Fair Truth-Spreader Placement Against Known Misinformation

Experiment Design v3: Prior Work, Method Roster, Metrics, and Full Pseudocode

Project working document

October 8, 2026

Contents

1 Overview **3**

2 Prior work, consolidated **3**

2.1 The six uploaded papers . . . 3

2.2 Related work found by search . . . 5

2.3 Scale of the field (impression, not a count) . . . 7

2.4 Gap this project targets . . . 7

3 Problem formalization and definitions **7**

4 Diffusion models **7**

5 Where to place the misinformation set M **8**

6 Experimental factors **9**

7 Metrics **9**

7.1 Six core metrics . . . 9

7.2 Secondary metrics . . . 10

8 How fairness is verified **10**

8.1 Built into the method . . . 10

8.2 Pass/fail checks . . . 10

8.3 Statistical checks . . . 11

8.4 Caveats . . . 11

8.5 What happens if protection concentrates on one group only . . . 11

9 Method roster **12**

10 Experiments **12**

11 Protocol, sanity checks, compute, deliverables **13**
11.1 Protocol . . . 13
11.2 Sanity tests . . . 14
11.3 Compute plan . . . 14
11.4 Deliverables . . . 14
11.5 Order of work . . . 14
11.6 Main risks . . . 15

12 Other diffusion models in the literature **15**

A Appendix: pseudocode for all algorithms **16**
A.1 Placing M . . . 16
A.2 Simulators (diffusion models) . . . 16
A.3 Bounds and fairness-agnostic baselines . . . 17
A.4 Individual-fair baselines (thesis) . . . 17
A.5 PROTECT and Naive PROTECT (thesis) . . . 18
A.6 Fast heuristics (Windham et al.) . . . 18
A.7 Group-fair baselines . . . 19
A.8 Fair truth-campaigning baselines from related work (sketches) . . . 20
A.9 Centrality baselines . . . 20
A.10 Multi-objective metaheuristic (Razaghi et al.) . . . 21
A.11 Proposed method: group-aware time-aware PROTECT . . . 21
A.12 Evaluation procedures . . . 22

1 Overview

Problem. A graph already contains a known set M of misinformation starters. We must place a budgeted set T of truth spreaders so that:

1. **Efficiency:** most vulnerable nodes are saved.
2. **Individual fairness:** no node is left badly protected (maximin / equality of save probabilities).
3. **Group fairness:** groups are protected evenly, both at the end of the cascade and across time.

Main welfare function (from the thesis, Maximin Protection):

$$\mu_{-\infty}(T) \;=\; \min_{v \in \text{vulnerable}} sp(v \mid T), \qquad sp(v \mid T) = 1 - \Pr[\, v \text{ infected by misinformation}\,].$$

Conventions. Items marked with a star* are my own design suggestions (not taken from the papers) and should be adjusted if you disagree. In the pseudocode appendix, "Thesis" means the pseudocode follows the thesis; "Sketch" means a simplified version that should be checked against the original paper before claiming a faithful reproduction.

2 Prior work, consolidated

This section gathers everything read or found so far in one place: the six papers you uploaded (read in full, except where noted), and related work found by web search (abstract or snippet level only, unless stated).

2.1 The six uploaded papers

1. Singh, Singh, Iyengar – Hybrid Centrality Measures for Service Coverage Problem (CSoNet 2019).

- **Problem:** choose one node for a service station so that the expected response time to demanding nodes is minimised. Demand is assumed proportional to betweenness (load).
- **Method:** two node-weighted hybrid centralities. HABC uses harmonic attenuation $1/(d+1)$ and EABC uses exponential attenuation α^d, both over betweenness weights. Cost $O(mn)$ unweighted.
- **Data and setup:** real flow networks (transport, energy, p2p, autonomous systems; 21 rows in the table although the text says 26); expected response time over $n/10$ requests.
- **Metrics:** expected response time; top-3 node rankings; Spearman rank correlation with degree, closeness, betweenness, eigenvector.
- **Findings:** no traditional measure is consistently best; at least one hybrid always minimises response time and beats all traditional ones on several metro networks; correlation with traditional measures is positive but small, lowest with degree.
- **Relevance here:** adapted as HABC_vuln / EABC_vuln, with initial vulnerability as the node weight.

2. Mohapatra, Singh, Singh, Saxena – Fairness-aware Analysis of Centrality Measures in Social Networks.

- **Problem:** are centrality-based seed sets fair to a minority in homophilic networks?

- **Setup:** 25 centralities (local, semi-local, global, modular). Homophily-BA networks (10,000 nodes, 80/20 groups, average degree 4, $h \in \{0.2, 0.4, 0.6, 0.8\}$, 10 graphs each); Facebook (4,039 nodes) and DBLP (16,501 nodes). Seed size 100. Intra-community probability 0.1; inter-community 0.1 / 0.05 / 0.025 (ICM1 uniform, ICM2 and ICM3 non-uniform).
- **Metrics:** influence outreach; Disparity (max gap in normalised outreach across communities); Influence Balance Ratio (largest over smallest community outreach); seed difference.
- **Findings:** the majority gains disproportionate influence, especially under non-uniform ICM, and bias grows with homophily. Fairness varies by individual centrality, not by category. The fairness-aware PageRanks (FSPR, LFPR) are the most equitable; LFPR gives higher outreach and FSPR greater fairness.

3. Windham et al. – Fast algorithms to improve fair information access in networks.

- **Problem:** Maximin Influence Maximization (maximise $\min_i \pi_i$), where the state of the art (Myopic) is bottlenecked by Monte Carlo estimation.
- **Contributions:** 10 new heuristics (BFS-, PPR- and topology-based), a corpus of 174 networks from 6 domains, *spreadability* for choosing α, the slope metric β, a fast ensemble and a random-forest meta-learner.
- **Metrics:** $\pi_{\min}$; β (slope of best-fit line of $\pi_{\min}$ vs. k, $k = 1..10$, averaged over about 20 runs); classification vs. Myopic (better / equivalent within one standard error / within 80% / worse); runtime and speed-up.
- **Findings:** Myopic has the best average performance but is not best everywhere; for 24% of networks (medium spreadability) no algorithm reaches 80% of Myopic. The meta-learner is on average about 20% worse than Myopic but roughly 76–133 times faster, and beats it on about 20% of networks. MinDegree_nd/ndn are 1000–10000 times faster than Monte Carlo methods. Speed-ups persist on networks of 34k and 87k nodes. Initialising with the highest-degree node gives a hidden advantage, so a random initial seed is used instead.

4. Fish et al. – Gaps in Information Access in Social Networks (WWW 2019).

- **Problem:** choose seeds so that the access gap between individuals is small; maximin welfare ($\min_v p_v$) instead of maximum reach.
- **Theory:** welfare functions modelled as ϕ-means. The rich get richer under all ϕ-means with $\phi < \infty$; all ϕ-means except maximin are $\Omega(n)$-imbalanced; maximin is balanced. Maximising the minimum probability is NP-hard even to approximate. Greedy, Myopic, Naive Myopic and an exact Gonzalez all have approximation ratios exponential in n.
- **Experiments:** six networks (EU, Arenas, Irvine, Facebook, ca-GrQc, ca-HepTh), k up to 100, $\alpha \in \{0.1, \ldots, 0.5\}$, 1000 simulations, 20 repeats; baselines TIM+ and Random.
- **Findings:** Myopic and Naive Myopic consistently best on minimum probability; maximin interventions correlate less with degree and centrality than reach-based ones; Myopic even beat TIM+ on average reach on three datasets, though it is about 10 times slower (e.g. 8.9 s vs. 0.9 s on Arenas at 100 seeds; Greedy 507 s).

5. Razaghi, Roayaei, Charkari – Group-Fairness-Aware Influence Maximization (IEEE TCSS 2023).

- **Problem:** extend Tsang et al.'s group fairness (maximin fairness, group rationality) with a *speed* of activation metric.
- **Method:** SetMOGWO, a discrete multi-objective grey wolf optimiser using only set operations; objectives are influence, maximin fairness, group rationality and activation speed.

- **Setup:** 24 networks of 500 nodes (Wilder et al., Antelope Valley); attributes age, gender, ethnicity (region dropped because values became zero); IC with $p = 0.1$, $k = 25$, population 500, 100 iterations, 30 runs.
- **Metrics:** influence; Maximin Fairness; Group Rationality (and mean violation); Activation Speed; Price of Fairness.
- **Findings:** outperforms Tsang et al.'s Algorithm DC and Maximin at lower price; Price of Fairness grows slowly with the number of objectives (at most 1.041 with four objectives); speed is the most expensive objective; fewer, more balanced groups (e.g. gender) give lower price; in SetMOGWO_IMRS influence falls 2.81% while maximin rises 58.7%, rationality 88.8% and speed 49.5%.

6. Unnikrishnan – Fairness in Negative Influence Minimization in Social Networks (Eindhoven master's thesis).

- **Problem:** truth campaigning against *known* misinformation starters; individual, feature-blind fairness. Introduces the **Maximin Protection** problem: utility is the save probability sp, welfare is the minimum $\mu_{-\infty}$.
- **Theory:** Maximin Protection is NP-hard and non-submodular; relationships between maximising average save probability, maximising saved nodes and fairness are established.
- **Methods:** PROTECT and Naive PROTECT (shortest-path information, matrix-based distance computation); baselines CMIA-O (fairness-agnostic), Greedy and Myopic Welfare Maximin (individual-fair), group-fair NIM methods.
- **Metrics:** saved fraction; average and maximin sp; Consistency (with a new vulnerability-based similarity); Gini coefficient.
- **Findings:** PROTECT beats all baselines on Maximin sp, by several folds on larger real networks. As average shortest path grows, PROTECT's lead over CMIA-O shrinks and CMIA-O gets the higher saved fraction; social networks usually have short paths, so PROTECT suits them.
- **Stated limitations:** Monte Carlo cost on large networks; a small but non-zero fairness cost with no trade-off parameter; weaker advantage on long-path networks.

2.2 Related work found by search

Items below were found by web search. For most I saw only the abstract, snippet or reference list; read the full papers before citing details.

Work	What it does	Use in this project
FWRRS – Saxena, Bierbooms, Pechenizkiy, "Fairness-aware fake news mitigation using counter information propagation" (Applied Intelligence, 2023)	Fairness-aware truth campaigning for known fake-news starters; weighted reversible reachable trees and maximin (community) fairness; reports up to about 10% more saved nodes and about 57% better maximin fairness than the second-best baseline.	**Main group-fair baseline** (not in the uploads).

Work	What it does	Use in this project
Scalable Fair Influence Blocking Maximization (arXiv 2601.22584)	First work on fair influence blocking under demographic-parity (DP) fairness; DP-aware objective combined with blocking effectiveness by tunable scalarisation.	Extra baseline and a DP group metric.
Socially Fair Mitigation of Misinformation via Constraint Stochastic Optimization (arXiv 2203.12537)	Fair allocation of mitigation budget as a stochastic, non-stationary knapsack, with a loss function for fairness among users.	Related; different (budget-allocation) view.
Fairness-aware edge blocking for influence minimization (Leiden master's thesis, 2024)	Fairness of influence minimisation by comparing saved-node proportions per affected community; disparity and maximin.	Related metric; edge blocking, not seeding.
FairSNA: Algorithmic Fairness in Social Network Analysis (Saxena et al., arXiv 2209.01678)	Survey; its fair-influence-blocking section states fairness there is "yet to be defined than being solved" and that fair-IM solutions do not transfer directly because the answer depends on the rumour starters.	Motivation and positioning.
Scalable Fair Influence Maximization (NeurIPS 2023)	Welfare fairness with an inequality-aversion exponent; reverse-influence-sampling algorithm.	Welfare-family sensitivity (μ_ϕ).
Reachability-Aware Fair Influence Maximization (2024)	Heuristics for the maximin-probability objective; up to 4× better fairness; no solution universally best across data, probability and propagation models.	Optional extra baseline; supports testing several models.
DQ4FairIM (arXiv 2512.00545)	Deep Q-learning for fair influence maximisation.	Optional learning-based baseline.
Minimizing the Influence of Misinformation via Vertex Blocking (arXiv 2302.13529)	Proves influence minimisation NP-hard and hard to approximate; GreedyReplace.	Background (vertex blocking, not truth seeding).
On Misinformation Containment in Online Social Networks (arXiv 1809.06486)	Multi-cascade model with priorities.	Diffusion model D5.
Misinformation Mitigation under Differential Propagation Rates and Temporal Penalties (arXiv 2206.11419)	Only the reference list was seen; the title suggests unequal propagation rates and time penalties.	Possibly relevant to MCICM and time fairness; check abstract.

Further titles seen only in reference lists, not read: DRIMUX, reverse prevention sampling, temporal influence blocking, StratLearner, hybrid sampling for misinformation prevention, node-blocking influence minimisation (VLDB 2024), and a poverty-reward concave fairness framework for influence maximisation (TKDD 2025).

2.3 Scale of the field (impression, not a count)

- Fair misinformation containment / fair truth campaigning: a handful of papers in my searches. Smallest area and closest to this project.
- Fair influence maximisation: moderate, a steady stream.
- Misinformation containment and influence blocking without fairness: large.
- Fairness-aware centrality: small.

A systematic review would need logged queries on arXiv, IEEE Xplore, ACM and Scopus; web search cannot enumerate these databases.

2.4 Gap this project targets

1. The thesis addresses individual fairness only; FWRRS addresses community maximin fairness. In my searches I found no work that evaluates *individual, group and time-based* fairness together for truth-spreader placement.
2. Most fair-IM and fair-blocking papers use one diffusion model. Windham et al. and the reachability-aware work show that rankings change across data and propagation models, so robustness across competitive models (COICM, MCICM, CLT, weighted cascade) is under-tested.
3. The thesis flags Monte Carlo cost and the lack of a fairness-cost trade-off parameter as limitations; fast heuristics (Windham et al.) and a Pareto view (Razaghi et al.) address these.
4. The "one group only" question (what if misinformation is stopped in one group) is, as far as I found, not systematically evaluated.

3 Problem formalization and definitions

- **Given:** $G = (V, E)$, group labels $C_1, \ldots, C_m$, known misinformation set M.
- **Choose:** truth set T, $|T| \leq k$, $T \cap M = \emptyset$.
- **Vulnerable set:** nodes reachable from M, excluding M. All metrics are computed on this set only.
- **Save probability** $sp(v \mid T)$: estimated by R Monte Carlo runs.
- **Group saved fraction:**

$$S_g \;=\; 1 - \frac{\mathbb{E}[\text{infected in } g \mid T]}{\mathbb{E}[\text{infected in } g \mid T = \emptyset]}.$$

  This is relative risk reduction per group; it avoids penalising a group that was barely threatened to begin with.

- **Generalised welfare (ϕ-mean):** $\mu_\phi = \left(\frac{1}{|V'|}\sum_v sp(v)^\phi\right)^{1/\phi}$, with $\phi = 1$ average, $\phi \to 0$ geometric, $\phi \to -\infty$ maximin. Used only as a sensitivity check.

4 Diffusion models

Every model returns, per node, the final state (negative / positive / inactive) *and the step at which it occurred*; the step is needed for time-fairness metrics.

ID	Model	Parameters	Role
D1	COICM (campaign-oblivious IC)	$p_T = p_M = p \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$	Core
D2	MCICM	$p_M \in \{0.1, 0.3, 0.5\} \times p_T \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ excluding equal pairs: 12 combinations (3 with $p_T < p_M$, 9 with $p_T > p_M$). $p_T = 1$ is the "high effectiveness" case on which the PROTECT rationale rests.	Core
D3	Competitive Linear Threshold (CLT)	Weight $1/\text{indeg}(v)^\star$, thresholds $\theta_v \sim U[0,1]$ resampled each run; optional truth multiplier $\gamma_T \in \{1, 1.5, 2\}^\star$. Submodular and monotonic (He et al.), so greedy has a $1 - 1/e$ guarantee.	Extended
D4	Weighted cascade	$p(u \to v) = 1/\text{indeg}(v)$, optional scale factors $c_T, c_M{}^\star$; no probability to tune.	Extended
D5	Multi-cascade with priorities	Per-node cascade priority $F_v(\cdot)$ acting as a heterogeneous tie rule.	Optional
D6	LT1DT	Linear threshold where a negative node can flip to positive on receiving correct information (one direction only).	Optional stress test

Variations applied on top of D1–D4.

- **Tie rule:** truth wins (default, as in the thesis), misinformation wins, or coin flip.
- **Detection delay** $r \in \{0, 1, 2, 3\}$**:** truth seeds activate r steps after misinformation. The PROTECT rationale assumes $r = 0$.

Probability regime. Besides the fixed grid, choose p by *spreadability* (Windham et al.): the p at which a random single seed of misinformation reaches about 20%, 50% and 80% of the network, so "low / medium / high spread" means the same thing on every graph.

Note on guarantees. MCICM is not submodular for arbitrary probabilities, but retains an approximation guarantee when the truth probability is 1 on every edge. Expect weaker guarantees in the $p_T < 1$ settings; the experiments should show this empirically.

5 Where to place the misinformation set M

M is fixed first as a labelled node set; then a baseline run with $T = \emptyset$ reveals what is vulnerable; only then is T chosen.

Size. $|M| = \lceil 0.03\, n \rceil^\star$, or fixed values ($|M| \in \{5, 10, 20\}$) on small graphs.

Mode	How M is chosen
highdeg	Top-$\lvert M \rvert$ nodes of the largest connected component by degree (ties by RNG).
random	Uniform sample from the largest connected component.
majority	Sample (or top-degree) inside the largest group only.
minority	Sample (or top-degree) inside the smallest group only.

Mode	How M is chosen
split_prop	Proportional to group sizes: $m_g = \text{round}(m\,\lvert C_g\rvert/n)$ per group.
split_equal	Equal number from each group.
clustered	First m nodes in BFS order from a random root.
dispersed	Farthest-point (Gonzalez) selection.

Rules.

- Fix M per (graph, seed) and give the *same* M to every method; draw 20 different M's (one per repeat) for random placements.
- Candidates for T are $V \setminus M$.
- M nodes start negative at step 0; T nodes start positive at step 0 (or step r under detection delay).
- For directed graphs, "reachable" means out-reachable from M.
- **Validation:** if fewer than 10% of nodes are vulnerable, redraw. If a group has no vulnerable nodes, mark it "unthreatened" and exclude it from group gap and ratio. Always report vulnerable-set size and group composition.

6 Experimental factors

Factor	Settings
Budget k	5, 10, 15, 20, 25, 30
M placement	highest-degree, random, all in majority, all in minority, split across groups
$\lvert M\rvert$	fixed, e.g. 3% of nodes*
Synthetic networks	Homophily-BA, 80/20 groups, $h \in \{0.2, 0.4, 0.6, 0.8\}$, 10 graphs per setting
Real networks	3–4 networks with an attribute defining groups; different degree and diameter
Monte Carlo	$R = 1000$ per estimate; 20 repeats with shared random seeds across methods

Two tiers to stay feasible.

- **Core grid:** D1 and D2, the full factor grid, all networks.
- **Extended grid:** D3, D4, tie rules and detection delay on one small and one medium network, with 3 budgets (5, 15, 30) and 2 M placements (highest-degree, "all in minority").

7 Metrics

7.1 Six core metrics

#	Metric	Definition	Better
1	Saved fraction	$\big(\mathbb{E}[\text{inf}, T{=}\emptyset] - \mathbb{E}[\text{inf}, T]\big)/\mathbb{E}[\text{inf}, T{=}\emptyset]$	higher
2	Maximin *sp* (welfare function)	$\min_{v \in \text{vuln}} sp(v)$	higher
3	Gini of *sp*	$\sum_{i,j} \lvert sp_i - sp_j \rvert \,/\, (2\,\lvert\text{vuln}\rvert \sum_j sp_j)$	lower
4	Consistency	$1 - \sum_{i,j} \lvert sp_i - sp_j \rvert\, s_{ij} / \sum_{i,j} s_{ij}$, with $s_{ij} = 1 - \lvert \text{vuln}_i - \text{vuln}_j \rvert$ and vulnerability = initial infection probability with $T = \emptyset$	higher
5	Group gap and ratio	gap $= \max_g S_g - \min_g S_g$; ratio $= \min_g S_g / \max_g S_g$	lower gap; ratio near 1
6	Time-weighted group protection	Per group, earlier saving scores higher (activation-speed idea of Razaghi et al.): $\sum_j n_{gj}(\text{max_it} - j + 1)/(\lvert C_g \setminus T \rvert \cdot \text{max_it})$; also max group gap over time and area between group infection curves	higher; smaller gaps

7.2 Secondary metrics

Average *sp*; **demographic parity gap** (difference of group saved fractions normalised by group size, following the DP notion of the fair influence blocking paper); leximin (compare sorted *sp* vectors); floor share (fraction of vulnerable nodes with $sp \geq \tau$); Price of Fairness; *sp* histogram and count of nodes near $sp \approx 0$; correlation of *sp* with degree, distance from M, and initial vulnerability; seed difference (majority seeds minus minority seeds); β (slope of maximin *sp* against k); runtime. **Sensitivity:** μ_ϕ for $\phi \in \{-\infty, -1, 0, 1\}$, and the inequality-aversion welfare family of Scalable Fair Influence Maximization (a weighted sum of concave powers of group saved fractions).

8 How fairness is verified

No theorem guarantees fairness here: maximin *sp* is NP-hard and the heuristics have poor worst-case ratios (Fish et al.). Fairness is therefore established three ways: in the objective, by measurement, and by statistical checks.

8.1 Built into the method

Maximin *sp* objective (individual); worst-off-group step (group); arrival-time tie-break (time).

8.2 Pass/fail checks

Thresholds must be fixed *before* running; the values below are placeholders⋆.

Check	Type	Pass condition
Maximin *sp* vs. agnostic baselines	Individual	higher, non-overlapping CIs
Floor share, $\tau = 0.5$	Individual	higher than baselines
Leximin	Individual	not worse than baselines
Gini and consistency	Individual	lower Gini, higher consistency
Correlation of *sp* with degree / distance from M	Structure bias	closer to 0 than baselines

Check	Type	Pass condition
Group gap and ratio	Group, final	gap ≤ 0.05, ratio ≥ 0.9
Max per-step group gap	Group, over time	$\leq \varepsilon_t$ (set in advance)
Group rationality	Group	no group worse off than with its proportional seed share

8.3 Statistical checks

1. **Independent evaluation runs:** select T with one set of simulations, evaluate on a fresh set. A minimum over noisy estimates is biased downward; evaluating on the selection runs would flatter every method.
2. **Bootstrap CIs** on maximin *sp* and the β-style classification (better / equivalent / within 80% / worse than a reference).
3. **Permutation test:** reshuffle group labels, keeping sizes, to test whether the observed group gap exceeds chance for that network structure.
4. **Paired tests:** shared seeds and Wilcoxon signed-rank across networks.
5. **Ablation** (experiment E13).

8.4 Caveats

- Individual and group fairness can conflict. Maximin *sp* does not guarantee group parity, and consistency does not detect group bias; always report both families.
- Group parity is not always the right target. If M lies entirely in group A and homophily is high, group B may have almost no vulnerable nodes; this is why group metrics use relative saved fraction.

8.5 What happens if protection concentrates on one group only

Metric	Behaviour	Why
Saved fraction	can look fine or good	group A may hold most vulnerable nodes or M
Average *sp*	rises	dominated by the protected group
Maximin *sp*	stays low	worst-off node is in the ignored group B
Group gap / ratio	large gap, low ratio	A well protected, B not
Gini	rises	save probabilities become unequal
Consistency	may not drop	similarity is feature-blind; it drops only if equally vulnerable nodes in A and B get different *sp*
Time curves	B's curve lags	truth reaches B late or never

Sweeping homophily separates the cases: at low h, ignoring B is costly because misinformation leaks across; at high h, a "firewall" around A may be enough.

9 Method roster

Group	Methods
Bounds	Random; no intervention ($T = \emptyset$)
Agnostic	CMIA-O; TopDegree; TIM+ (adapted to saved nodes); SetSOGWO
Individual-fair	Greedy Welfare Maximin (small graphs only); Myopic Welfare Maximin; Naive Myopic
Thesis	PROTECT; Naive PROTECT
Fast heuristics	Gonzalez; MinDegree_hc / hcn / nd / ndn; LeastCentral / LeastCentral_n; Myopic BFS / Naive Myopic BFS; Myopic PPR / Naive Myopic PPR
Ensemble	Fast ensemble (oracle); meta-learner
Group-fair	**FWRRS** (Saxena et al., main fair truth-campaigning baseline); **DP-aware fair blocking** (arXiv 2601.22584); Group Maximin Greedy (Tsang-style); Algorithm DC; FSPR; LFPR
Optional extras	Reachability-aware fair IM heuristics; DQ4FairIM; GreedyReplace (vertex blocking, as a non-fair blocking reference)
Centrality	PageRank; Betweenness; Bridging; MMB (modular); K-shell; HABC_vuln / EABC_vuln
Multi-objective	SetMOGWO (adapted)
Ours	Group-aware time-aware PROTECT

About 33 methods. Run all of them only on the small and medium networks; on larger networks run the simulation-free methods plus PROTECT, Myopic, FSPR/LFPR, SetMOGWO (reduced iterations) and the proposed method.

Adaptation notes. TIM+, HABC_vuln and SetMOGWO were not designed for this problem; describe them as "adapted" in any write-up. HABC_vuln replaces the betweenness weight of Singh et al. with initial vulnerability $1 - sp_0(v)$.

10 Experiments

ID	Question	Run	Key outputs
E1	Which method gives the best welfare?	All methods × D1/D2 × all parameter settings × $k = 5..30$	Maximin vs. k, with saved fraction, Gini, consistency
E2	Does the intervention just follow network structure?	Correlate sp with degree, distance from M, initial vulnerability	Correlation per method; sp histogram; nodes near 0
E3	Which cheap method is almost as good as the slow one?	β per method and network; classification vs. reference; best method vs. average degree; runtime vs. size; ensemble and meta-learner	β table; classification heatmap; runtime plot
E4	How does homophily change group outcomes?	Sweep $h = 0.2..0.8$ × five M placements; real networks	Group gap/ratio; seed difference; fairness-vs-efficiency scatter

ID	Question	Run	Key outputs
E5	What does fairness cost, and does it respect timing?	PoF; % loss in saved fraction vs. % gain in maximin, group ratio, time-weighted protection; effect of number/imbalance of groups; per-step group curves	PoF tables; group infection curves
E6	Can cheap centrality replace simulation?	Centrality seeds, HABC_vuln/EABC_vuln, FSPR/LFPR vs. PROTECT	Spearman of rankings; Jaccard overlap of seed sets; six core metrics
E7	Do seeds transfer across models?	Select T under one model, evaluate under all others (D1–D4, tie rules, delay)	Transfer matrix; rank correlation of method rankings
E8	What if protection concentrates on one group?	T all in A, all in B, balanced; across M placements and homophily	All six metrics; group saved fraction and maximin
E9	Is the result an artifact of maximin?	Recompute with μ_ϕ, $\phi \in \{-\infty, -1, 0, 1\}$ and the inequality-aversion welfare family	Method ranking per ϕ and per aversion parameter
E10	Do conclusions hold across model families?	Repeat E1 under D3 (with γ_T variation) and D4; optionally D5, D6	Rankings and metrics per family
E11	Do tie rule and detection delay matter?	Vary tie rule and $r \in \{0, 1, 2, 3\}$ under D1 and D3	Performance vs. r, esp. PROTECT's distance rule
E12	What does the multi-objective trade-off look like?	SetMOGWO Pareto front vs. single-point methods on saved fraction, maximin, group ratio, time-weighted protection	Pareto plots; one rule-based operating point
E13	Does each part of the proposed method matter?	Remove group step, then time step, then both	Metric deltas; paired tests
E14	How do we compare with fair truth-campaigning prior work?	FWRRS, DP-aware fair blocking, PROTECT and the proposed method on the same M, models and budgets; report individual, group and time metrics together	Head-to-head table; which fairness notion each method wins on

Expectations to confirm or refute.

- E1/E3: fair heuristics beat agnostic ones on maximin at a modest saved-fraction cost.
- E10/E11: PROTECT's distance rule is strongest at $p_T = 1$, $r = 0$, truth-wins; it may weaken under CLT, delay, or a misinformation-wins tie rule.
- E4/E8: for agnostic methods, seeds drift toward the majority as homophily rises.

11 Protocol, sanity checks, compute, deliverables

11.1 Protocol

1. Generate graphs, groups, and M for every placement; fix all random seeds.

2. **Baseline run** with $T = \emptyset$: store $mp_0(v)$, the vulnerable set, the similarity matrix and baseline group curves; reuse for all methods.
3. For each (graph, M, model, parameters, tie rule, delay, k, method): select T (selection runs); evaluate on fresh independent simulations logging per-step per-group counts; compute all metrics.
4. Repeat 20 times; report means, standard errors, bootstrap CIs.
5. Run the fairness checks and sanity tests.

11.2 Sanity tests

- Random should be worst on maximin.
- Greedy should be best on tiny graphs.
- Every metric should improve (or stay equal) as k grows.
- At $p_T = 1$, $r = 0$, truth-wins, PROTECT's distance rule should agree with simulation.
- With $T = \emptyset$, saved fraction must be 0.
- For CLT, sp stays in $[0, 1]$ and the simulator is consistent under truth-wins.

11.3 Compute plan

- Greedy Maximin only on graphs of a few hundred nodes.
- Simulation-free methods (Naive PROTECT, Gonzalez, MinDegree, LeastCentral, centralities, HABC_vuln) on the full grid.
- Start with $R = 1000$ and 10 repeats; raise to 20 for final plots.
- Cache cascades: evaluation runs can be shared across methods with the same (graph, M, model, seed).
- Core grid size is roughly 17 probability settings $\times$ 6 budgets $\times$ 5 placements $\times$ about 30 methods $\times$ graphs $\times R$; hence the two-tier plan.

11.4 Deliverables

Tables: metrics by method per model; classification vs. reference; transfer matrix (E7); pass/fail summary per method.
Figures: maximin vs. k per model; fairness-vs-efficiency scatter; homophily sweep; per-group infection curves over time; sp histograms; best method vs. average degree; Pareto front (E12); performance vs. detection delay (E11).

11.5 Order of work

1. Simulator: D1 and D2 with per-step per-group logging, then D3/D4, tie rules, delay; add `EstimateSP` and the metrics module.
2. Baselines: Random, TopDegree, Myopic, Greedy, then PROTECT and Naive PROTECT; validate on a small graph.
3. Run E1–E3 on small and medium graphs.
4. Add group-aware and group-fair methods (including FWRRS); run E4, E5, E8, E14.
5. Add remaining fast heuristics, ensemble and centralities; run E6, E9.

6. Extended grid: E7, E10, E11, E12, E13, E14.
7. Scale up and run final statistical checks.

11.6 Main risks

- **Compute:** large grid; mitigated by the two-tier plan and cached cascades.
- **Noise in minimum values:** handled by independent evaluation runs and bootstrap CIs.
- **Adapted methods:** TIM+, HABC_vuln, SetMOGWO are not designed for this problem.
- **CLT parameterisation:** the weight and multiplier choices are mine, not the original papers'; state them explicitly.
- **Pass/fail thresholds:** placeholders; fix them before any results are seen.
- **FWRRS and DP-aware blocking:** only abstract-level descriptions were read; the pseudocode here is a sketch. Use the authors' code or full papers for a faithful comparison.

12 Other diffusion models in the literature

Beyond COICM and MCICM (Budak et al.), the literature offers the following competitive models.

- **Competitive Linear Threshold (He et al.):** submodular and monotonic, so hill climbing gives a $1 - 1/e$ approximation.
- **Multi-cascade model with priorities:** each user ranks cascades by priority; supports more than two campaigns.
- **Competitive Activation Model (Thai et al., CSoNet 2015):** designed for the containment problem with known misinformation sources.
- **LT1DT (Li and Giua):** models people who first adopt a rumour and later change their mind on seeing correct information.
- **Trust-aware dynamic LT:** adds trust and time variation to competitive propagation.
- **Point processes (e.g. Hawkes):** continuous-time alternative to IC/LT.

Own variations*: weighted cascade; tie rule as a parameter; detection delay r.

A Appendix: pseudocode for all algorithms

Notation. G, M, T, k; alpha (or pT, pM); R = Monte Carlo runs; D = shortest-distance matrix (computed with M's incoming edges removed); D_M[v] = min over m in M of D[m][v]; vulnerable = nodes with D_M finite, excluding M. A candidate c is an *enhancer* of node v when D[c][v] <= D_M[v] (truth can reach v no later than misinformation). Lines starting with # are comments.

A.1 Placing M

```
PlaceM(G, groups, m, mode, seed):
    rng <- RandomGenerator(seed)
    pool <- nodes of the largest (strongly) connected component
    if mode = "highdeg":      M <- top-m of pool by degree (ties by rng)
    if mode = "random":       M <- rng.sample(pool, m)
    if mode = "majority":     g <- largest group;  M <- rng.sample(pool & C_g, m)
    if mode = "minority":     g <- smallest group; M <- rng.sample(pool & C_g, m)
    if mode = "split_prop":   for each group g: m_g <- round(m*|C_g|/n);
                                  M <- M + rng.sample(pool & C_g, m_g)
    if mode = "split_equal":  for each group g: sample m/#groups nodes from pool & C_g
    if mode = "clustered":    r <- rng.choice(pool); M <- first m nodes in BFS order from r
    if mode = "dispersed":    M <- Gonzalez farthest-point selection of m nodes in pool

    # validation with a baseline run (T = empty)
    sp0 <- EstimateSP(G, M, {}, model, R)
    vulnerable <- { v not in M : sp0[v] < 1 }
    if |vulnerable| < 0.1*n or vulnerable is empty: redraw with a new seed
    for each group g: record |vulnerable & C_g| and baseline infection of g
        # if a group has no vulnerable nodes: mark "unthreatened",
        # exclude it from group gap / ratio
    return M, vulnerable, sp0
```

A.2 Simulators (diffusion models)

```
SimulateIC(G, M, T, pT(u,v), pM(u,v), tie_rule, delay r):   # D1, D2, D4
    # D1 COICM: pT = pM = alpha.  D2 MCICM: pT != pM.
    # D4 weighted cascade: p(u,v) = c / in_degree(v).
    state[v] <- inactive; time[v] <- INF
    for m in M: state[m] <- negative; time[m] <- 0
    pos_frontier <- {}; neg_frontier <- M; step <- 0
    while neg_frontier or pos_frontier nonempty or step < r:
        step <- step + 1
        if step = r: activate T as positive (time = r); pos_frontier <- pos_frontier + T
        new_pos <- {}; new_neg <- {}
        for u in pos_frontier, v in neighbors(u) with state[v] = inactive:
            if rand() < pT(u,v): new_pos <- new_pos + {v}
        for u in neg_frontier, v in neighbors(u) with state[v] = inactive:
            if rand() < pM(u,v): new_neg <- new_neg + {v}
        for v in new_pos & new_neg: resolve by tie_rule (truth / misinfo / coin)
        set states and times; update frontiers
    return state, time          # with r = 0, T is activated at step 0
```

```
SimulateCLT(G, M, T, weights w(u,v), gammaT, tie_rule, delay r):   # D3
    theta[v] <- Uniform(0,1) for every v          # resampled each run
    start as in SimulateIC
    repeat each step:
        for each inactive v:
            W_pos <- gammaT * sum of w(u,v) over positive in-neighbors u
            W_neg <- sum of w(u,v) over negative in-neighbors u
            if W_pos >= theta[v] and W_neg >= theta[v]: resolve by tie_rule
            else if W_pos >= theta[v]: v <- positive
            else if W_neg >= theta[v]: v <- negative
    until no change
    # w(u,v) = 1 / in_degree(v) is the default used here

SimulatePriority(G, M, T, priority F_v):                    # D5 (sketch)
    as SimulateIC, but when several cascades reach v in the same step,
    v adopts the cascade with the highest priority F_v(cascade)
```

```
SimulateLT1DT(G, M, T, theta, theta2):                    # D6 (sketch)
    as SimulateCLT, plus one rule: a negative node v flips to positive
    if its positive weight >= theta2[v] (one direction only)
```

```
EstimateSP(G, M, T, model, R):                  # thesis Algorithm 1
    count[v] <- 0
    repeat R times:
        state, time <- model(G, M, T)
        for v with state[v] = negative: count[v] += 1
    return sp[v] = 1 - count[v] / R

Preprocess(G, M):
    G' <- G with all incoming edges of M removed
    D  <- all-pairs shortest distance on G'      # about O(c*|V|^3) via matrix powers
    D_M[v] <- min over m in M of D[m][v]
    vulnerable <- { v not in M : D_M[v] < INF }
    return D, D_M, vulnerable
```

A.3 Bounds and fairness-agnostic baselines

```
Random(G, M, k):      return k nodes sampled uniformly from V \ M
TopDegree(G, M, k):   return the k nodes of V \ M with the highest degree
NoIntervention:       T = {}
```

```
CMIA_O(G, M, k, alpha, theta):                  # sketch
    # build local arborescences of paths with misinformation probability >= theta
    T <- {}
    repeat k times:
        for c in V \ (M + T):
            gain[c] <- expected reduction in infected nodes over all arborescences if c added
        T <- T + { argmax gain[c] }
    return T

TIM_plus_adapted(G, M, k, alpha, num_sets):     # sketch (reverse sampling)
    sets <- {}
    repeat num_sets times:
        pick a random node v in V \ M
        sample a live-edge graph under the model
        RR <- { c : truth seeded at c would reach v before misinformation does }
        sets <- sets + {RR}
    T <- greedy maximum coverage: repeat k times, pick the node in the most uncovered sets
    return T

SetSOGWO(V \ M, k, pop, iters, objective = saved fraction):    # sketch
    same procedure as SetMOGWO below but with a single objective and no archive;
    the leaders alpha, beta, delta are the three best wolves
```

A.4 Individual-fair baselines (thesis)

```
GreedyWelfareMaximin(G, M, k, alpha, R):
    vulnerable <- V \ M; candidates <- V \ M; T <- {}
    repeat k times:
        for c in candidates:
            sp <- EstimateSP(G, M, T + {c}, alpha, R)
            min_sp[c]  <- min over vulnerable of sp
            avg_sp[c]  <- mean over vulnerable of sp
            min_cnt[c] <- |{ v : sp[v] = min_sp[c] }|
        best <- max over c of min_sp[c]
        cand <- { c : min_sp[c] = best }
        if |cand| > 1: cand <- { c in cand : min_cnt[c] is smallest }
        if |cand| > 1: cand <- { c in cand : avg_sp[c] is largest }
        t <- cand[0]; T <- T + {t}; candidates <- candidates - {t}
    return T

MyopicWelfareMaximin(G, M, k, alpha, R):
    vulnerable <- V \ M; T <- {}
    repeat k times:
```

```
        sp <- EstimateSP(G, M, T, alpha, R)
        t <- argmin over vulnerable of sp        # ties random
        T <- T + {t}; vulnerable <- vulnerable - {t}
    return T

NaiveMyopic(G, M, k, alpha, R):
    sp <- EstimateSP(G, M, {}, alpha, R)
    return the k nodes of V \ M with the lowest sp
```

A.5 PROTECT and Naive PROTECT (thesis)

```
PROTECT(G, M, k, eps, alpha, R):
    D, D_M, vulnerable <- Preprocess(G, M)
    D_T[v] <- -INF for v in vulnerable; candidates <- V \ M; T <- {}
    repeat k times:
        sp <- EstimateSP(G, M, T, alpha, R)
        min_sp <- min over vulnerable of sp
        worst <- { v in vulnerable : sp[v] = min_sp }
        cnt[c] <- |{ v in worst : D[c][v] <= D_M[v] }|
        options <- { c : cnt[c] = max cnt }
        if |options| > 1 and max cnt = |worst|:          # look-ahead
            next <- { v in vulnerable : sp[v] <= min_sp + eps }
            cnt[c] <- |{ v in next : D[c][v] <= D_M[v] }| for c in options
            options <- { c : cnt[c] = max cnt }
        if |options| > 1:                                # widest protection
            for c in options:
                D_Tc[v] <- min(D[c][v], D_T[v])
                cnt[c] <- |{ v in vulnerable : D_Tc[v] <= D_M[v] }|
            options <- { c : cnt[c] = max cnt }
        if |options| > 1:                                # earliest arrival
            options <- { c in options : mean over v of D_Tc[v] is smallest }
        t <- options[0]; T <- T + {t}
        D_T[v] <- min(D[t][v], D_T[v]); candidates <- candidates - {t}
    return T
```

```
NaiveProtect(G, M, k, eps):
    D, D_M, vulnerable <- Preprocess(G, M)
    n_M[v] <- |{ m in M : D[m][v] = D_M[v] }|
    n_T[v] <- 0; candidates <- V \ M; T <- {}
    target <- { v in vulnerable : D_M[v] = 1 }
    while |T| < k and target nonempty:
        if min over target of n_T = 0:   diff[v] <- n_M[v] if n_T[v] = 0 else -INF; eps' <- 1
        else:                            diff[v] <- (n_M[v] - n_T[v]) / n_M[v];      eps' <- eps
        worst <- { v in target : diff[v] = max diff }
        cnt[c] <- |{ v in worst : D[c][v] <= D_M[v] }|
        options <- { c : cnt[c] = max cnt }
        if |options| > 1 and max cnt = |worst|:
            next <- { v in target : diff[v] >= max diff - eps' }
            cnt[c] <- |{ v in next : D[c][v] <= D_M[v] }| for c in options
            options <- { c : cnt[c] = max cnt }
        if |options| > 1:
            cnt[c] <- |{ v in vulnerable : D[c][v] <= D_M[v] }|
            options <- { c : cnt[c] = max cnt }
        t <- options[0]; T <- T + {t}; candidates <- candidates - {t}
        n_T[v] <- |{ s in T : D[s][v] = D_M[v] }|
        target <- target - { v : n_T[v] >= n_M[v] }
    if |T| < k:
        fill T with the (k - |T|) candidates c maximizing
            |{ v in vulnerable : D[c][v] <= D_M[v] }|
    return T
```

(The thesis pseudocode has misplaced `end` markers; the structure above is the intended one.)

A.6 Fast heuristics (Windham et al.)

A random initial seed is used and removed from the final T. Candidates always exclude M.

```
Gonzalez(G, M, k):
```

```
    T <- { random node in V \ M }
    repeat k times:
        t <- argmax over c in V \ (M + T) of  min over s in T of dist(c, s)
        T <- T + {t}
    return T minus the initial seed

MinDegree_hc(G, M, k):               # picks the node itself
    T <- { random node in V \ M }
    repeat k times:
        pool <- { v not in T+M : deg(v) = min degree among non-seeds }
        x <- argmin over pool of harmonic_centrality(v)
        T <- T + {x}
    return T minus the initial seed

MinDegree_hcn:  as MinDegree_hc, but add y = argmax over u in neighbors(x) of deg(u)
MinDegree_nd:   pool as above; x <- argmax over pool of sum of deg(w), w in neighbors(v); add x
MinDegree_ndn:  as MinDegree_nd, but add x's highest-degree neighbor

LeastCentral(G, M, k):
    T <- { random node in V \ M }
    repeat k times:
        t <- argmin over v not in T+M of closeness(v); T <- T + {t}
    return T minus the initial seed
LeastCentral_n: add the highest-degree neighbor of t instead of t
```

```
BFS_Estimate(G, M, T, alpha):
    for each seed s (latest first):
        BFS from s; for node v at layer t:
            pi_hat[v] <- probability v is reached from its parents (layer t-1)
                         and its same-layer neighbors, using alpha
    return pi_hat (updated across seeds)

MyopicBFS(G, M, k, alpha):
    T <- { random node in V \ M }; pi_hat <- BFS_Estimate(G, M, T, alpha)
    repeat k times:
        t <- argmin over v not in T+M of pi_hat[v]; T <- T + {t}
        pi_hat <- BFS_Estimate(G, M, T, alpha)
    return T minus the initial seed
NaiveMyopicBFS: run BFS_Estimate once, take the k lowest

MyopicPPR(G, M, k):
    T <- { random node in V \ M }
    repeat k times:
        ppr <- PersonalizedPageRank(G, restart_set = T)
        t <- argmin over v not in T+M of ppr[v]; T <- T + {t}
    return T minus the initial seed
NaiveMyopicPPR: compute PPR once from the initial seed, take the k lowest
```

```
FastEnsemble_oracle(G, M, k, E):          # E = 5 chosen heuristics
    for a in E: T_a <- a(G, M, k); score[a] <- beta of T_a (via EstimateSP)
    return T_a for the a with the highest score[a]

SelectEnsemble(corpus, reference):
    E <- {}
    repeat 5 times:
        add the algorithm that most increases the number of networks where some
        member of E reaches >= 80% of the reference method's performance

MetaLearner(G, M, k, E, trained_model):
    features <- (n, avg degree, max degree, degree variance, clustering,
                 avg shortest path, diameter, assortativity)
    a <- trained_model.predict(features)   # random forest over E
    return a(G, M, k)
# training: one label per training network = best algorithm in E (by beta); 80/20 split
```

A.7 Group-fair baselines

```
GroupMaximinGreedy(G, M, k, alpha, R, groups):      # simple stand-in
    T <- {}
    repeat k times:
        for c in V \ (M + T):
```

```
            sp <- EstimateSP(G, M, T + {c}, alpha, R)
            for each group g: S_g[c] <- group saved fraction
            score[c] <- min over g of S_g[c]
        T <- T + { argmax score[c] }
    return T

TsangMaximin(G, M, k, groups):                      # sketch of the idea only
    keep a weight w_g per group (multiplicative weights)
    repeat for several rounds:
        S <- greedy maximize sum_g w_g * (nodes saved in g)/|C_g| with budget k
        raise w_g for groups that did worse; lower it for groups that did better
    combine the solutions and round to a k-subset
# use the authors' algorithm if reproducing their results exactly

AlgorithmDC(G, M, k, groups):                       # sketch
    k_g <- ceil(k*|C_g|/n)
    require each group's saved count >= what it would reach seeding only its own
    subgraph G[C_g] with k_g seeds (diversity constraint)
    maximize total saved nodes subject to those constraints

FSPR(G, k, target ratios phi_g):                    # sketch
    choose a group-wise jump vector so each group's total PageRank mass equals phi_g
    (bisection on the jump weights); return top-k by score, excluding M
LFPR(G, k, phi_g):                                  # sketch
    at each step each node distributes rank so the share reaching each group
    follows phi_g; return top-k by score, excluding M
# use the authors' implementation (Tsioutsiouliklis et al.) for real runs
```

A.8 Fair truth-campaigning baselines from related work (sketches)

Only abstract-level descriptions of these were read. The pseudocode captures the stated idea; use the original papers for exact steps.

```
FWRRS(G, M, k, groups):                    # Saxena et al. -- sketch of the stated idea
    build weighted reversible reachable trees rooted at vulnerable nodes
        (a tree records which candidate seeds can reach the root before misinformation)
    for each candidate c and each community g:
        block_power[c][g] <- weighted count of g's trees that c would protect
    T <- {}
    repeat k times:                        # the paper describes six selection steps
        g* <- community with the lowest current protected fraction  (maximin fairness)
        choose c maximising the fairness-aware objective, favouring block_power[c][g*]
            and breaking ties by total block_power, then by coverage of other communities
        T <- T + {c}; update the trees covered by c
    return T
```

```
DP_aware_FIBM(G, M, k, groups, lambda):   # arXiv 2601.22584 -- sketch of the stated idea
    effectiveness(T) <- expected number of saved nodes
    dp_fair(T)       <- demographic-parity term: how close the group saved fractions are
    objective(T)     <- effectiveness(T) + lambda * dp_fair(T)    # tunable scalarisation
    T <- greedy maximisation of objective(T) with budget k
          (the paper proves an approximate-monotonic-submodular structure that
           makes a scalable greedy / sampling-based solution possible)
    return T
```

```
InequalityAversionWelfare(S_1..S_m, weights w_g, alpha):    # welfare-family check
    # S_g = group saved fraction; alpha near 1 favours total spread,
    # alpha near 0 favours fairness (Scalable Fair Influence Maximization)
    return sum over g of w_g * (S_g ^ alpha)

DemographicParityGap(T, groups):
    return max over g of S_g - min over g of S_g, with S_g normalised by group size
```

A.9 Centrality baselines

```
CentralityTopK(G, M, k, measure):
    score[v] <- measure(G, v)       # PageRank, Betweenness, Bridging, MMB, K-shell, ...
```

```
    return top-k of V \ M by score

HABC_vuln(G, M, k, sp0):            # adapted from Singh et al.
    w[v] <- 1 - sp0[v]              # initial vulnerability (0 for M)
    for u in V \ M:
        score[u] <- ( w[u] + sum over v != u of w[v]/(d(u,v)+1) ) / sum over x of w[x]
    return top-k of score
EABC_vuln: replace 1/(d(u,v)+1) with a^d(u,v), a in {0.5, 0.75}
```

A.10 Multi-objective metaheuristic (Razaghi et al.)

```
SetMOGWO_adapted(V \ M, k, pop_size, max_it, t):
    for i in 1..pop_size: X_i <- random k-subset of V \ M; V_i <- V \ M
    archive <- {}
    for it in 1..max_it:
        evaluate each X_i:  f1 = saved fraction,  f2 = maximin sp,
                            f3 = min over groups of group saved fraction,
                            f4 = time-weighted group protection
        update archive with non-dominated solutions
        choose leaders X_alpha, X_beta, X_delta from the archive (selection strategy)
        a <- 2 - it*(2/max_it)
        for each wolf X_i:
            if |V_i| < t*k: V_i <- V \ M
            compute r1, r2, A, C as in MOGWO
            X_leader <- closest leader to X_i            # set distance = |X_i \ X_leader|
            if |A| < 1:                                  # exploitation
                EN <- X_leader \ X_i; EO <- X_i \ X_leader; D <- |EO|
                step <- min(ceil((1-|A|)*D), |EO|, |EN|)
                X_i <- (X_i - rand(EO, step)) + rand(EN, step)
            else:                                        # exploration
                EN <- V_i \ (X_i + X_leader); EO <- X_leader & X_i; D <- |X_i \ X_leader|
                step <- min(ceil((|A|-1)*D), |EO|, |EN|)
                V_step <- rand(EN, step)
                X_i <- (X_i - rand(EO, step)) + V_step
                V_i <- V_i - V_step
        update archive with the new non-dominated solutions
    return archive      # Pareto front; pick one point by a pre-set rule
```

A.11 Proposed method: group-aware time-aware PROTECT

```
GroupAwareProtect(G, M, k, eps, alpha, R, groups, lambda):
    D, D_M, vulnerable <- Preprocess(G, M)
    Vg[g] <- vulnerable & C_g
    sp0 <- EstimateSP(G, M, {}, alpha, R)
    base_inf[g] <- sum over Vg[g] of (1 - sp0[v])
    D_T[v] <- -INF; T <- {}; candidates <- V \ M
    repeat k times:
        sp <- EstimateSP(G, M, T, alpha, R)
        for each group g with base_inf[g] > 0:
            saved[g] <- 1 - (sum over Vg[g] of (1 - sp[v])) / base_inf[g]
        g* <- argmin over g of saved[g]                  # worst-off group
        min_sp <- min over Vg[g*] of sp
        worst <- { v in Vg[g*] : sp[v] = min_sp }
        cnt[c] <- |{ v in worst : D[c][v] <= D_M[v] }|
        options <- { c : cnt[c] = max cnt }
        if |options| > 1 and max cnt = |worst|:
            next <- { v in Vg[g*] : sp[v] <= min_sp + eps }
            options <- options maximizing |{ v in next : D[c][v] <= D_M[v] }|
        if |options| > 1:
            options <- options maximizing |{ v in vulnerable : min(D[c][v], D_T[v]) <= D_M[v] }|
        if |options| > 1:                                # time-aware tie-break
            options <- options minimizing
                mean over v in Vg[g*] of min(D[c][v], D_T[v])
                + lambda * spread across groups of that mean
        t <- options[0]; T <- T + {t}; candidates <- candidates - {t}
        D_T[v] <- min(D[t][v], D_T[v])
    return T
```

A.12 Evaluation procedures

```
ComputeMetrics(G, M, T, groups, model, R_eval):
    # use FRESH simulations, independent of those used to select T
    sp, time, group_curves <- R_eval runs of model with T, M
    vulnerable, sp0 <- from the baseline run
    saved_fraction <- (sum(1-sp0) - sum(1-sp)) / sum(1-sp0)       # over vulnerable
    maximin        <- min over vulnerable of sp
    gini           <- sum_ij |sp_i - sp_j| / (2*|vuln|*sum(sp))
    consistency    <- 1 - sum_ij |sp_i - sp_j|*s_ij / sum_ij s_ij,  s_ij = 1-|vuln_i - vuln_j|
    S_g            <- 1 - infected_g(T) / infected_g(empty) per group
    group_gap      <- max_g S_g - min_g S_g
    group_ratio    <- min_g S_g / max_g S_g
    time_protect_g <- sum_j n_gj*(max_it - j + 1) / (|C_g \ T| * max_it)   # earlier is better
    floor_share    <- fraction of vulnerable v with sp[v] >= tau
    leximin_vec    <- sorted(sp)
    corr           <- correlation of sp with degree, D_M, vulnerability
    return all metrics
```

```
Beta(method, G, M, model):                # slope of maximin vs budget (Windham et al.)
    repeat 20 times:
        points <- [ (k, maximin(method(G, M, k))) for k in budgets ]
        beta_i <- slope of the least-squares line through points
    return mean(beta_i), standard error

Classify(method, reference):              # per network
    if beta_method > beta_ref + SE:      "better"
    elif |beta_method - beta_ref| <= SE: "equivalent"
    elif beta_method >= 0.8*beta_ref:    "within 80%"
    else:                                "worse"

Spreadability(G, model, grid of p, R):    # choose p for 20% / 50% / 80% spread
    for p in grid: f(p) <- mean fraction of nodes reached from a random single seed over R runs
    return the p values where f(p) is closest to 0.2, 0.5, 0.8

PermutationTestGroupGap(T, sims, groups, n_perm = 1000):
    obs <- group_gap on the true labels
    repeat n_perm times:
        shuffle group labels (keeping sizes); null_i <- group_gap on shuffled labels
    return p-value = fraction of null_i >= obs

PriceOfFairness(method):  PoF <- best saved fraction among all methods / saved fraction of method
```